Assignment 8: Basic SQL Queries and Aggregate Functions

DBMS Assignment Series | Project topic: **Online Food Delivery System (QuickBite)**

Name: **Jagan S** | Roll No.: **2501121240031**

1. Objective

Write SQL queries to retrieve and manipulate data in the QuickBite database, demonstrating WHERE, ORDER BY, GROUP BY, HAVING and the aggregate functions COUNT(), SUM(), AVG(), MIN() and MAX(). Each query is shown with its actual output and a brief explanation. Data used is the sample data from Assignment 7.

2. Filtering records using WHERE

2.1. WHERE with AND / comparison - Vegetarian dishes costing less than Rs. 150.

```
SELECT item_name, category, price
FROM menu_items
WHERE is_veg = 1 AND price < 150
ORDER BY price;
```

Output (12 rows):

item_name	category	price
Filter Coffee	Beverage	40
Idli Sambar	Starter	70
Lemon Iced Tea	Beverage	80
Gulab Jamun	Dessert	90
Cold Coffee	Beverage	99
French Fries	Starter	99
Masala Dosa	Main Course	110
Classic Veg Burger	Main Course	120
Mango Smoothie	Beverage	120
Garlic Bread	Starter	129
Rasmalai	Dessert	130
Chocolate Shake	Beverage	140

Explanation: WHERE keeps only rows satisfying both conditions; AND requires both to be true (is_veg = 1 means vegetarian).

2.2. WHERE with LIKE and OR - Regular customers whose name starts with 'A' or 'M'.

```
SELECT full_name, area, membership
FROM customers
WHERE membership = 'Regular' AND (full_name LIKE 'A%' OR full_name LIKE 'M%');
```

Output (2 rows):

full_name	area	membership
Aditya Rao	Indiranagar	Regular
Mohan Raj	Whitefield	Regular

Explanation: LIKE 'A%' matches any value beginning with A. Parentheses make sure OR is evaluated before the AND with membership.

2.3. WHERE with IS NULL - Orders that never got a delivery partner.

```
SELECT order_id, customer_id, order_date, status
FROM orders
WHERE partner_id IS NULL;
```

Output (2 rows):

order_id	customer_id	order_date	status
6	5	2026-08-23	Cancelled
18	11	2026-10-07	Cancelled

Explanation: NULL means 'no value', so it is tested with IS NULL (never = NULL). These are the cancelled orders.

2.4. WHERE with IN and BETWEEN - Delivered or placed orders made in October 2026 paid by UPI or Card.

```
SELECT order_id, order_date, status, payment_method
FROM orders
WHERE order_date BETWEEN '2026-10-01' AND '2026-10-31'
  AND status IN ('Delivered', 'Placed')
  AND payment_method IN ('UPI', 'Card')
ORDER BY order_date;
```

Output (10 rows):

order_id	order_date	status	payment_method
15	2026-10-01	Delivered	UPI
16	2026-10-03	Delivered	UPI
17	2026-10-05	Delivered	Card
19	2026-10-09	Delivered	UPI
20	2026-10-12	Delivered	Card
21	2026-10-14	Delivered	UPI
22	2026-10-16	Delivered	UPI
24	2026-10-22	Placed	UPI
25	2026-10-24	Placed	Card
26	2026-10-25	Placed	UPI

Explanation: BETWEEN gives an inclusive date range and IN tests membership in a list of values.

3. Sorting records using ORDER BY

3.1. ORDER BY ... DESC - Five most expensive menu items (ties sorted by name).

```
SELECT item_name, price
FROM menu_items
ORDER BY price DESC, item_name ASC
LIMIT 5;
```

Output (5 rows):

item_name	price
Pepperoni Pizza	399
Butter Chicken	320
Margherita Pizza	299
Chicken Chettinad	280
Chilli Chicken	260

Explanation: DESC puts the highest price first; the second sort key breaks ties. LIMIT 5 keeps the top five rows.

3.2. ORDER BY on a date - Five most recent orders.

```
SELECT order_id, customer_id, order_date, status
FROM orders
ORDER BY order_date DESC, order_id DESC
LIMIT 5;
```

Output (5 rows):

order_id	customer_id	order_date	status
26	7	2026-10-25	Placed
25	1	2026-10-24	Placed
24	9	2026-10-22	Placed

order_id	customer_id	order_date	status
23	8	2026-10-19	Delivered
22	4	2026-10-16	Delivered

Explanation: Dates in YYYY-MM-DD form sort correctly, so DESC returns the newest orders first.

4. Grouping records using GROUP BY

4.1. GROUP BY with COUNT and AVG - Number of dishes and average dish price per restaurant (by restaurant_id).

```
SELECT restaurant_id, COUNT(*) AS dishes, ROUND(AVG(price), 2) AS avg_price
FROM menu_items
GROUP BY restaurant_id
ORDER BY avg_price DESC;
```

Output (6 rows):

restaurant_id	dishes	avg_price
3	4	231.5
1	4	217.5
4	4	175.0
6	4	142.5
5	4	134.75
2	4	125.0

Explanation: GROUP BY collapses all dishes of a restaurant into one row; COUNT(*) counts them and AVG() averages their prices.

4.2. GROUP BY on orders - How many orders each customer has placed.

```
SELECT customer_id, COUNT(*) AS orders_placed
FROM orders
GROUP BY customer_id
ORDER BY orders_placed DESC, customer_id;
```

Output (11 rows):

customer_id	orders_placed
1	5
3	3
4	3
7	3
2	2
5	2
6	2
8	2
9	2
10	1
11	1

Explanation: One row per customer. Customer 12 has never ordered, so she does not appear (an outer join in Assignment 9 shows such customers).

4.3. GROUP BY on status - Orders by status.

```
SELECT status, COUNT(*) AS number_of_orders
FROM orders
GROUP BY status
ORDER BY number_of_orders DESC;
```

Output (3 rows):

status	number_of_orders
Delivered	21

status	number_of_orders
Placed	3
Cancelled	2

Explanation: Splits the orders into Delivered, Placed and Cancelled and counts each group.

5. Filtering grouped results using HAVING

5.1. HAVING with COUNT - Customers who have placed 3 or more orders.

```
SELECT customer_id, COUNT(*) AS orders_placed
FROM orders
GROUP BY customer_id
HAVING COUNT(*) >= 3
ORDER BY orders_placed DESC, customer_id;
```

Output (4 rows):

customer_id	orders_placed
1	5
3	3
4	3
7	3

Explanation: WHERE filters individual rows before grouping; HAVING filters the groups after aggregation, so it can use COUNT().

5.2. HAVING with AVG - Restaurants whose average dish price is above Rs. 150.

```
SELECT restaurant_id, ROUND(AVG(price), 2) AS avg_price
FROM menu_items
GROUP BY restaurant_id
HAVING AVG(price) > 150
ORDER BY avg_price DESC;
```

Output (3 rows):

restaurant_id	avg_price
3	231.5
1	217.5
4	175.0

Explanation: Only restaurant groups whose computed average exceeds 150 are kept.

5.3. WHERE + GROUP BY + HAVING together - Customers with at least 2 delivered orders.

```
SELECT customer_id, COUNT(*) AS delivered_orders
FROM orders
WHERE status = 'Delivered'
GROUP BY customer_id
HAVING COUNT(*) >= 2
ORDER BY delivered_orders DESC, customer_id;
```

Output (7 rows):

customer_id	delivered_orders
1	4
3	3
4	3
2	2
6	2
7	2
8	2

Explanation: Order of work: WHERE removes cancelled/placed orders, GROUP BY forms customer groups, HAVING keeps groups with two or more rows.

6. Aggregate functions: COUNT(), SUM(), AVG(), MIN(), MAX()

6.1. All five aggregates on order_items - Sales summary of all order lines.

```
SELECT COUNT(*)                      AS order_lines,
       SUM(quantity)                 AS total_units,
       SUM(quantity * unit_price)    AS item_revenue,
       ROUND(AVG(unit_price), 2)     AS avg_unit_price,
       MIN(unit_price)               AS cheapest_item,
       MAX(unit_price)               AS costliest_item
FROM order_items;
```

Output (1 row):

order_lines	total_units	item_revenue	avg_unit_price	cheapest_item	costliest_item
52	69	11417	175.19	40	399

Explanation: COUNT(*) counts rows, SUM() adds units and revenue (quantity x price), AVG() gives the mean price, MIN()/MAX() give the extremes. Revenue here includes cancelled orders.

6.2. Aggregates grouped by payment method - Delivery fees and activity by payment method.

```
SELECT payment_method,
       COUNT(*)                 AS orders,
       SUM(delivery_fee)        AS total_fees,
       ROUND(AVG(delivery_fee), 2) AS avg_fee,
       MIN(order_date)          AS first_order,
       MAX(order_date)          AS last_order
FROM orders
GROUP BY payment_method
ORDER BY orders DESC;
```

Output (3 rows):

payment_method	orders	total_fees	avg_fee	first_order	last_order
UPI	14	310	22.14	2026-08-02	2026-10-25
Card	7	160	22.86	2026-08-09	2026-10-24
Cash	5	150	30.0	2026-08-05	2026-10-19

Explanation: Each payment method becomes one row with its own count, total, average and earliest/latest order date (MIN/MAX also work on dates).

6.3. COUNT(DISTINCT ...) - How many different customers and restaurants appear in the orders.

```
SELECT COUNT(*)                     AS total_orders,
       COUNT(DISTINCT customer_id)   AS distinct_customers,
       COUNT(DISTINCT restaurant_id) AS distinct_restaurants,
       COUNT(partner_id)             AS orders_with_partner
FROM orders;
```

Output (1 row):

total_orders	distinct_customers	distinct_restaurants	orders_with_partner
26	11	6	24

Explanation: DISTINCT removes repeats before counting. COUNT(column) skips NULLs, so COUNT(partner_id) counts only orders that have a partner.